Serverless Architecture for Scalable Digital Voucher Processing

Sérgio Vinício da Silva Oliveira
Student ID: 23170981
Scalable Cloud Computing, MSc in Cloud Computing
National College of Ireland Dublin, IRELAND
Email: x23170981@student.ncirl.ie. URL: www.ncirl.ie
Client-side deployed app URL: https://ftqnaqgrig.execute-api.eu-west-1.amazonaws.com/dev
Admin-side deployed app URL: https://mdsfx9mm14.execute-api.eu-west-1.amazonaws.com/dev
API Specification (Swagger): URL: https://app.swaggerhub.com/apis/student-c67-be7/api-safetrade-voucher/1.0.0

***Abstract*—This project describes a Proof of Concept (PoC) that serves as validation for an event-driven and scalable architecture leveraging AWS in a digital voucher management context. The primary goal is to overcome performance bottlenecks by implementing auto-scaling solutions and asynchronous workflows. The architecture refers to the separation of responsibilities layer, the approach implemented focus in eliminating blocking operations during peak demand and keeping user response times low through AWS Lambda Functions, DynamoDB and AWS SQS, and also refers to an infrastructure deployment as DevOps process that allows repeatable and reliable provisioning. To validate scalability, load tests were conducted by creating and consuming vouchers in the range of 1 to 5000 under requests in the exposed API. As a result, the average response time remained at approximately 90 ms without significant variation across all scenarios, where is demonstrated that the decoupled design aligns with principles of scalability besides maintains stable latency under higher workloads.**

I. Introduction

Digital voucher systems have become a commonplace in current online commerce and learning platforms. However, traditional monolithic architectures typically have limited ability to scale performance when dealing with a high volume of concurrent requests. In a synchronous model, the server has to wait for heavy processing tasks before a transaction can be resolved. This creates a bottleneck that inhibits horizontal scalability and increases latency, particularly during peak demand.

The context behind this project is resolving these limitations using the Concepts of Scalable Cloud Programming, by moving away from a synchronous approach. This approach separates the business logic from the asset processing pipeline, enabling the core application to stay responsive while heavy loads are asynchronously handled by auto-scaling serverless components.

The overall aim of this PoC is to showcase a cloud-native architecture that is ready to scale. In addition it uses a CI/CD pipeline to deploy the infrastructure, ensuring provisioning is repeatable, automated and follows cloud scaling principles. The rest of the paper is organized as follows: In section II, is presented the Project Specification and functional requirements. Next, Section III describes the architecture design and event-driven workflow. In Section IV the implementation will be presented. After in Section V, its elaborate on CI/CD pipeline and DevOps integration. Section VI summarizes the outcomes learned and a short reflection of the work.

II. Project Specification

This project aims a distributed Web3 design. It provides two different applications, ensuring that there is a separation of concerns between the client and administrator.

A. The Front-end Application

- **Admin-Side App:** It is used for managing and monitoring the vouchers.
- **Client-Side App:** Users can purchase vouchers through the MetaMask wallet on this side.

B. Web Services and Integration

In order to provide high availability and interoperability, the application interacts with five different services:

- **[Ownership API]:** AWS API Gateway in a AWS Lambda Service, which is responsible for receiving client requests vouchers demand.
- **[Classmate API] QRcode Generation API:** This is an external microservice that a peer developed, which provides a QRcode image.
- **[Alchemy API]:** Used to confirm the transaction hashes on Sepolia Ethereum testnet.
- **Cloud Storage (S3):** An object storage QRcode images.
- **NoSQL Database (DynamoDB):** A database that stores the state of vouchers.

C. Back-End Logic and Scalability Mechanisms

The backend infrastructure consists of two distinct AWS Lambda functions that are configured for horizontal scaling and asynchronous processing:

- **Verification Lambda (Synchronous):** Engages with the client-side app to trigger the Alchemy API. It confirms the transaction on the chain. Once is confirmed it performs updates in the voucher status in DynamoDB.

- **Processor Lambda (Asynchronous):** This function is triggered by DynamoDB Streams and implements a Change Data Capture pattern and also invokes the API of a classmate.

III. Architecture and Design

This section explains the architecture of the proposed system and the technical choices made to guarantee scalability, loose coupling, and resilience.

The 1 illustrates the complete system architecture, including internal and external services integrations. All choices discussed in this section are represented in this unified diagram. Rather than describing each interaction step by step, the following sections examine the same architecture from different technical perspectives focused in a scalable solution.

A. Structural Overview

Fig. 1 presents cloud-native architecture composed of client interfaces, API management, compute services, persistence layer and asynchronous processing components.

The interaction level of the system is divided into two applications:

- **Client-Side:** Purchase of vouchers.

- **Admin-Side:** Voucher management and monitoring

Both communicate through AWS API Gateway endpoints. At the compute level, the architecture is built on stateless AWS Lambda functions. Persistence is handled by DynamoDB, while object storage is managed by Amazon S3. External integrations include the Alchemy API (blockchain validation) and a QR Code generation external API. Fig. 1 shows a structural organization that separates all these concerns.

Additionally, to promote the same service as a component to other applications, an API implemented in AWS Lambda has been exposed via API Gateway. As presented in Fig. 2, this service provides a POST endpoint that returns a voucher ID stored in DynamoDB managed by the admin-side app, the fully documentation is find on Swagger [1]. This reinforces an API-first design and demonstrates interoperability between distributed services.

B. Concurrency and Processing Model

From a concurrency perspective, Figure 1 separates the synchronous transaction path from the asynchronous processing path. The synchronous path includes:

- API Gateway

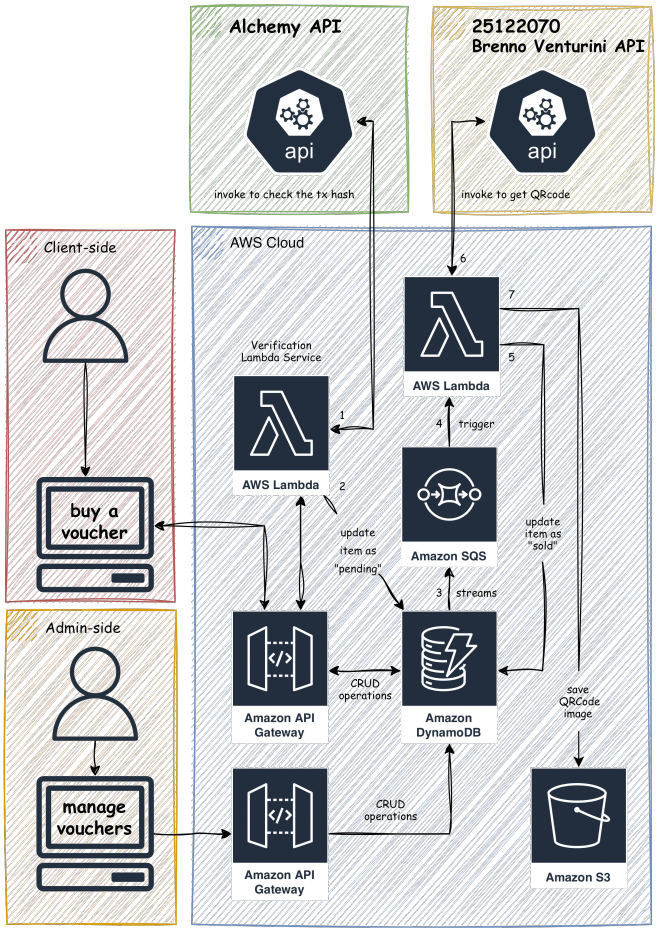

Fig. 1. App Architecture Diagram

- AWS Lambda (Verification)
- DynamoDB

This path focus on validations transactions and update voucher state. No heavy asset generation is executed during this stage. By keeping this layer lightweight, the system reduces execution time and allows higher concurrency without vertical scaling.

The asynchronous path includes:

- DynamoDB Streams
- Amazon SQS
- AWS Lambda (Processor)

This path is triggered after state persistence. Once a voucher record is updated, DynamoDB Streams emits a change event. This event is delivered to Amazon SQS, which acts as a buffering and decoupling layer before invoking the Processor Lambda. This separation follows a non-blocking design model, in others words, this approach ensures that user expirence are not blocked by heavier background tasks, which is a fundamental concept in Scalable Cloud Programming.

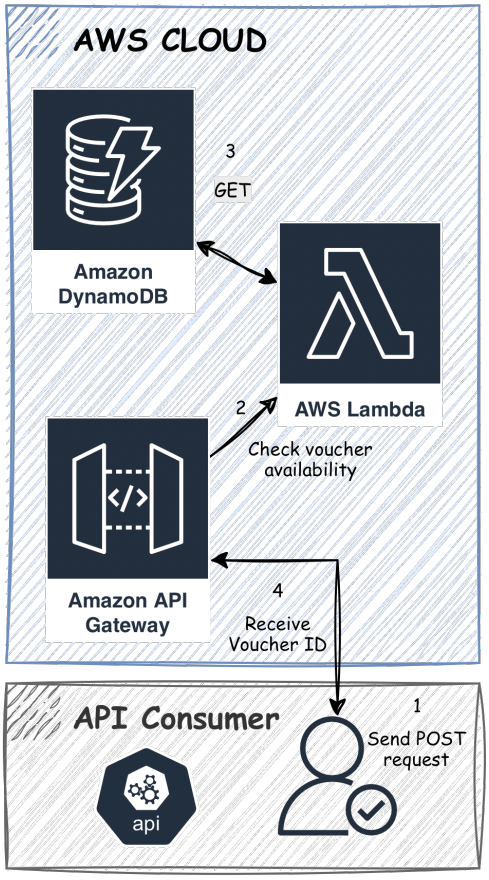

Fig. 2. API Architecture Diagram

C. Decoupling and Event Propagation Strategy

Figure 1 highlights that there is no direct invocation between the Verification Lambda and the Processor Lambda. The communication occurs indirectly through DynamoDB Streams and Amazon SQS. This design implements a Change Data Capture (CDC) pattern, which enables the system to react to state changes in the database instead of including asset generation logic in the transaction flow. This approach provides:

- Loose coupling between components
- Independent scaling of processing units
- Clear separation between write operations and background tasks
- Reduced risk of cascading failures

A producer-consumer model is introduced by using SQS, which is a standard base to distributed systems, where the database update acts as the producer of the event, while the processor's Lambda consumes messages asynchronously, managing sudden workloads, in addition to maintaining system stability. By allocating responsibilities instead of increasing the computational capacity of a single component. This ensures the desire scalability through the decoupling strategy, as presented in Figure 1.

D. Resilience and Scalability Considerations

The architectural boundaries visible in Figure 1 also support fault isolation. The queue layer acts as a control boundary between transaction confirmation and asset generation. If QR code generation fails, the transaction remains confirmed in DynamoDB. The message stays in SQS and can be retried. Because Lambda functions are stateless and independently scalable, the system can increase processing capacity automatically based on workload. There is no shared memory or tight dependency between components. This design improves:

- Horizontal scalability
- Elastic behavior under demand spikes
- Failure containment
- Operational stability

The separation between synchronous and asynchronous responsibilities is a key factor in scaling in this architecture.

E. Final Note on Architectural Rationale

Figure 1 serves as the central representation of the system. Each subsection analyzed the same structure under different technical lenses: structural organization, concurrency model, decoupling strategy and resilience boundaries. This layered interpretation demonstrates that the architecture aligns with key principles of Scalable Cloud Programming, particularly event-driven processing, stateless compute elasticity and distributed workload management.

IV. Implementation

This section describes the implementation of the proposed architecture, services and supporting tools described in Section III.

A. Serverless Backend Deployment

The back-end infrastructure allows for an automatic horizontal scaling mode according to incoming traffic, is divided into two distinct execution patterns within the AWS Lambda ecosystem. ThisThe differentiated by its responsibility and invocation mode:

- **Core Apps (Client-side and Admin-side)**
  These two applications follow the monolithic serverless model, where the business logic, and user interface is centralised in a full Django Python Application. This approach uses the Zappa framework to encapsulate and deploy the applications in the AWS Lambda. In addition an API Gateway is included as a universal router for each side, routing all HTTP requests to a single Lambda function that manages routes internally.
  The deployment of both are automated via CI/CD through GitHub Actions to ensure reproducibility and abstraction of the infrastructure configuration. The details will be further discussed in section xxxxx.

- **Specialized Service Layer (Atomic Functions)**
  Unlike the core applications, these functions are decoupled and perform granular tasks.

– **Verification Lambda:** Acts as a validation middleware. It is invoked via API Gateway to perform communication with the Alchemy API and persistence on Amazon DynamoDB.
– **Lambda Processor (Asynchronous/Event-Driven):** Operating in the background after database updates, this function is responsible for integration with external QR Code Services API developed by Brenno Venturini - x25122070@student.ncirl.ie [2], and and persistence of assets in Amazon S3.

B. Database Configuration (DynamoDB)

For voucher state management, Amazon DynamoDB was configured as the data store. In addition DynamoDB Streams was configured to capture item-level changes. The stream configuration captures new and updated records, which are used to trigger the asynchronous workflow as show in a Fig. 3 captured from AWS Lambda Function. No complex relational modeling was required, as the system operates mainly on single-item lookups and updates, where through this approach the performance is optimised for high write velocity scenarios.

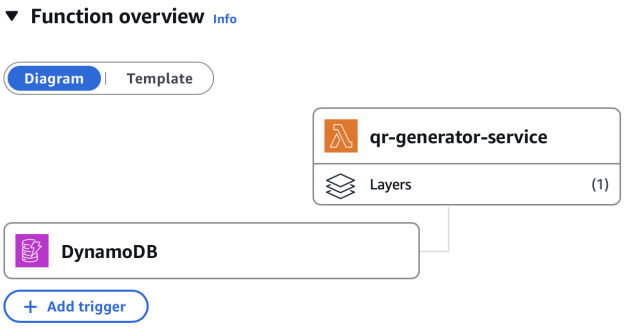

Fig. 3. DynamoDB Streams to Lambda Function Diagram

C. Asynchronous Processing Setup

To implement the decoupled processing layer, the stream connected to an Amazon SQS.

Processor Lambda is configured with SQS as event source as presented in Fig. 4. This approach helps in keeping the time-consuming tasks like external API invocation separate from transaction confirmation, thus ensuring that synchronous transaction confirmations are not impacted by heavy operations. In addition, batch processing config had been done moderately to keep a balance between throughput and execution time. This also avoids long-running Lambdas and provides better fault isolation.

D. External API Integration

The system integrates with two external services:

- **Alchemy API** used to validate transaction hashes on the Sepolia Ethereum testnet [3].
- **QR Code Generation API** external microservice developed by Brenno Venturini - moodle@ncirl.ie [2].

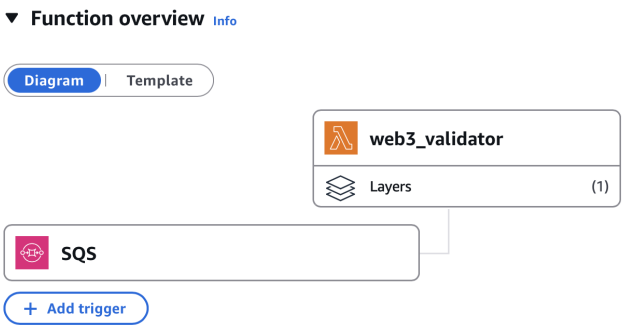

Fig. 4. SQS trigger to Lambda Function Diagram

E. Public API Exposed

This project implements a separate public API to serve other systems. This API, is exposed via API Gateway and is implemented in AWS Lambda, documented using Swagger (OpenAPI specification) [1]. This service provides a POST endpoint without parameters that returns a generated voucher ID stored in DynamoDB. The listing 1 shows business logic.

```
def lambda_handler(event, context):
    try:
        # get an item using GSI
        dynamo_response = table.query(
            IndexName="voucher_status_index",
            KeyConditionExpression=boto3.dynamodb
            .conditions.Key('voucher_status')
            .eq("Active"),
            Limit=1
        )
        item = dynamo_response.get('Items', [])

        #no vouchers avaialible
        if not item:
            return {'statusCode': 404,
            'body': json.dumps('Sorry! We have
                sold out our vouchers')}

        voucher = item[0]
        voucher_id = voucher['voucher_id']

        # update the voucher to "Sold"
        table.update_item(
            Key={
                'voucher_id': voucher_id
            },
            UpdateExpression="SET
                voucher_status = :status",
            ExpressionAttributeValues={
                ':status': "Gifted"},
            ReturnValues="UPDATED_NEW"
        )
        return {'statusCode': 200,
        'body': json.dumps(f"Tks for using
            SafeTrade - Vouchers API. Your
            voucher ID is '{voucher_id}'")}

    except ClientError as e:
        return {'statusCode': 409,
            'body': json.dumps(f"Sorry! This item
                {voucher_id} is no longer
                avialiable anymore")}
```

Listing 1. API Implementation

F. Monitoring and Observability

To verify runtime behavior monitoring performed using:

- **AWS CloudWatch** - Metrics and logs for Lambda.
- **DynamoDB metrics** - Read/write capacity behavior.
- **Postman** - Perform requests and get response time.

Metrics gathered in controlled experiments confirmed stable response times and consistent Lambda execution behavior with increasing request batches. Performance analysis is detailed in Section V.

V. Scalability and Evaluation

The section provides empirical evidence to validate the proposed architecture. This project aim a PoC where tests were executed to evaluate the system performance with increasing loads. The objective was to check for stability of responses, performance of database and scalability of API.

A. Test Design

The tests performed were structured in different batches, in the context of voucher creation by using the Admin-side Application, and also request has been performed through the Postman by using the API URL. Through this, AWS Lambda execution and DynamoDB operational metrics have been observed.

The test performed did not focus on the stress test to failure, but rather on consistency and scalability behavior with progressive load increase.

B. API Response Stability

The public voucher API was tested using POST requests through the Postman in different quantities. The Figure 5 shows the invocations performed.

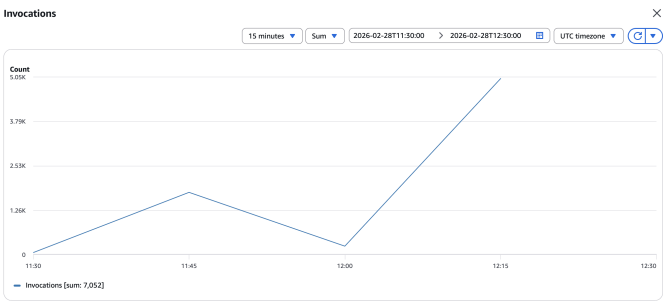

Fig. 5. Invocations captured from AWS Lambda Function

As mentioned earlier by Fig. 5, a different number of requests were made, however, as shown in the Fig. 6, it is possible to identify the consistency of the mean value of 71.99 milliseconds for all cases.

From another angle, Figure 7 shows the average response time recorded by Postman during the 5000 request test. This indicates that:

- Lambda scaled automatically in response to demand.

Fig. 6. Duration captured from AWS Lambda Function

- The asynchronous path was decoupled from the synchronous execution path.

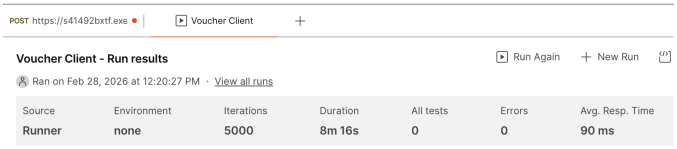

Fig. 7. 5000 POST Request performed through Postman

C. DynamoDB Write Behavior

In order to test database stability, voucher creation tests were run in incremental batches as presented in Fig. 8, without manual scaling intervention.

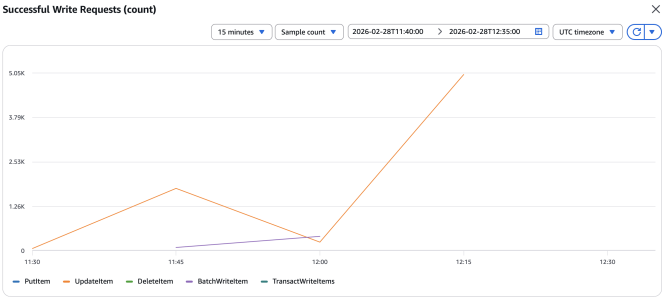

Fig. 8. Successful Write Request - AWS DynamoDB

DynamoDB was able to across all the workloads tested, stable write performance. The result was less than 1 ms, and is presented in Figure 9

Since the table was set up in on-demand mode, capacity automatically scaled with request rate. The write latency variation is zero too, confirming that our data have been horizontal partitioned well enough to handle the increased volume.

The results validate that the chosen data model is scalable for volume and velocity without degradation of performance in moderate growth scenarios.

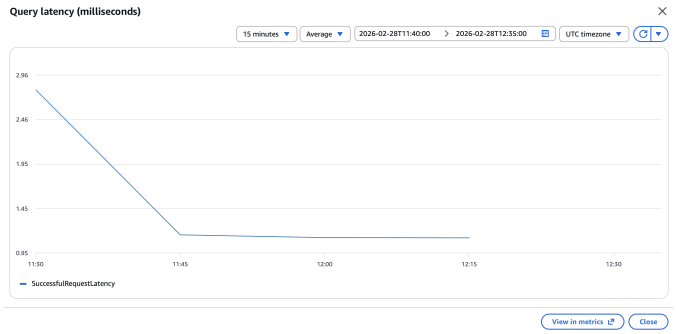

Fig. 9. Latency during the write test - AWS DynamoDB

VI. Continuous Integration, Delivery and Deployment

This section covers the CI/CD approach used in this project. The goal of the pipeline is to maintain code quality and continuous deployment.

The CI/CD workflow using GitHub Actions is demonstrated in Figure 10.

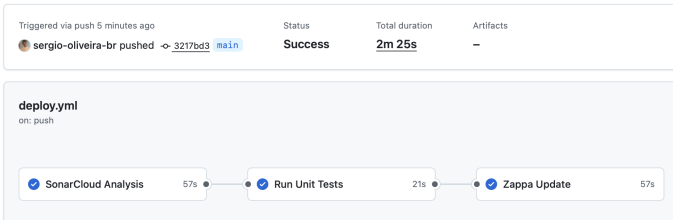

Fig. 10. Pipeline CI/CD - GitHub Actions [4]

The pipeline is automatically triggered on every push to the main branch and consists of three sequential stages, Static code analysis (SonarQube), Automated unit tests and Serverless deployment using Zappa.

The stages are run in order and deployment only happens if all previous steps have been successful. The first step does SonarQube Analysis to identify possible vulnerabilities and maintainability problems. Automated tests are run after static analysis. The project contains limited unit tests, this step ensures core functionality and guards against regression during updates. This approach during the deployment pipeline increase both the reliability of the systems and minimize the risk of unstable code releases. Following analysis and tests passing, the pipeline triggers automatic deployment through Zappa. The deployment stage includes, python environment setup, dependency installation, AWS credential configuration, and execution of Zappa update dev command.

This updates all of the Lambda functions and API Gateway configuration automatically.

VII. Conclusion and Reflection

This project presents a resilient event-driven architecture to manage digital vouchers on AWS. This propose design focus on responsive interaction by decoupling synchronous user interactions from asynchronous processing.

The integration of DynamoDB Streams and Amazon SQS allowed for efficient Change Data Capture (CDC) patterns to decouple processing and horizontally scale. The approach achieved a consistent response times (90 ms) even through scalability tests with high volume (up to 5000 request).

The usage of AWS Lambda as the execution layer for the client-side and administrative services, allows to auto-scale on demand while reducing infrastructure management. The DynamoDB its support of high-velocity workloads, flexible data modeling, and horizontal scaling.

Predictable deployments in a CI / CD pipeline with GitHub Actions and automated tests implemented, follows the best practices from cloud native development and principles for Scalable Cloud Programming.

In conclusion, the proposed architecture presents an elasticity approach through the market leaders with a decoupled, serverless, event-driven architecture.

References

[1] Sérgio Oliveira. (2026) Safetrade - vouchers api. Accessed: 2026-02-22. [Online]. Available: https://app.swaggerhub.com/apis/student-c67-be7/api-safetrade-voucher/1.0.0

[2] Brenno Venturini. (2026) ????? Accessed: 2026-02-18.

[3] Alchemy. (2026) Infrastructure that moves billions at scale. Accessed: 2026-02-18. [Online]. Available: https://www.alchemy.com

[4] Sérgio Oliveir. (2026) Github actions - pipeline ci/cd 70. Accessed: 2026-03-02. [Online]. Available: https://github.com/sergio-oliveira-br/safetrade_admin_service/actions/runs/22593616259